Continuous Fairness On Data Streams

Abstract

We study the problem of enforcing continuous group fairness over windows in data streams. We propose a novel fairness model that ensures group fairness at a finer granularity level (referred to as block) within each sliding window. This formulation is particularly useful when the window size is large, making it desirable to enforce fairness at a finer granularity. Within this framework, we address two key challenges: efficiently monitoring whether each sliding window satisfies block-level group fairness, and reordering the current window as effectively as possible when fairness is violated. To enable real-time monitoring, we design sketch-based data structures that maintain attribute distributions with minimal overhead. We also develop optimal, efficient algorithms for the reordering task, supported by rigorous theoretical guarantees. Our evaluation on four real-world streaming scenarios demonstrates the practical effectiveness of our approach. We achieve millisecond-level processing and a throughput of approximately 30,000 queries per second on average, depending on system parameters. The stream reordering algorithm improves block-level group fairness by up to 95% in certain cases, and by 50–60% on average across datasets. A qualitative study further highlights the advantages of block-level fairness compared to window-level fairness.

ACM Reference Format:

. 2018. Continuous Fairness On Data Streams. In . ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Many real-world applications inherently generate data streams rather than static datasets - for example, financial market tickers, performance metrics in network monitoring and traffic management, server logs, content recommendation engines, and user click streams in web analytics and personalization systems [3, 14]. Due to the continuous and evolving nature of data streams, querying them requires continuous queries, i.e. queries that are executed persistently over time and return updated results as new data arrive. A fundamental concept in stream processing are the sliding windows allowing to analyze continuously flowing data in real time.

In parallel, fairness [7] in algorithmic decision-making has emerged as a critical issue in data management, especially in high-stakes domains such as hiring, lending, and law enforcement [13, 22, 30, 32, 34]. A substantial body of research has focused on *group fairness*, which seeks to ensure that algorithmic outcomes do not systematically disadvantage particular demographic groups. However, most of this work assumes access to static datasets, where fairness constraints are applied once on a fixed input. Despite the widespread relevance of these streaming scenarios in real world applications, the study of fairness in streaming settings remains significantly underexplored.

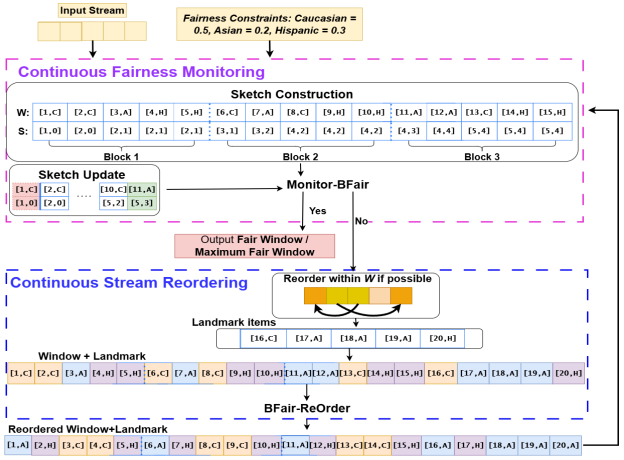


Figure 1: The proposed framework consists of two key components: *a. Continuous fairness monitoring* checks group fairness constraints per block of each sliding window. b. *Continuous stream reordering* reorders the current window plus the landmark items when needed

Motivating Application - Online Content Recommendation

In a news or video streaming platform that continuously recommends content in real time, items arrive from creators across different ethnic groups (e.g., Caucasian, Asian, Hispanic). The system displays this content in paginated form, where each sliding window of recommendations is divided across multiple pages.

Even when the overall window includes a balanced mix of content from all groups, the page-wise distribution may still be skewed — for example, the first few pages may predominantly feature Caucasian creators, while later pages contain more balanced or diverse representation.

Since users' attention typically drops sharply across pages — with most engagement concentrated on the first one or two — this ordering bias results in unequal visibility and exposure. Thus, ensuring proportional representation across an entire window is insufficient; fairness must also be enforced within and across pages, accounting for both the presentation order and attention decay patterns.

Contributions. In this paper, for the first time we study group fairness algorithms on streaming data. We introduce a novel fairness model that partitions each sliding window into multiple blocks and enforces fairness at the block level (Figure 1 contains three such blocks). These blocks, naturally defined by the application (e.g., page), provide a localized and flexible alternative to traditional notions of fairness that operate only at the window level or based on prefixes. When an application restricts each window to a single block, our formulation naturally reduces to the classical window-level group fairness model.

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/XXXXXXX.XXXXXXX

To support this framework, we study two interrelated problems: **Continuous Fairness Monitoring** and the **Continuous Stream Reordering Problem**. The monitoring module continuously checks whether fairness constraints hold within each block of every sliding window (see Continuous Fairness Monitoring in Figure 1). When a violation is detected, the system first attempts to reorder the items within the current window to restore fairness. If reordering alone cannot resolve the issue, the decision making pauses - it reads a small number of additional items—referred to as landmark bits—which effectively extend the current stream and create new sliding windows. Conceptually, these landmarks represent a limited "buffer time" that the application is willing to tolerate before producing output (see Continuous Stream Reordering in Figure 1). The framework then performs reordering over both the current window and the landmark bits. We formalize this task as the *Continuous Stream Reordering Problem*, aimed at maximizing the number of distinct fair blocks across both current and newly spawned slided windows. Once the reordering is done, **Continuous Fairness Monitoring** outputs the fair window.

We define a **sketch-based** data structure to pre-process every window and an Algorithm `Monitor-BFair` for answering *Monitoring Continuous Fairness*. The sketch compactly encodes protected attribute distributions within each block of a sliding window, enabling exact and efficient fairness evaluation. The sketch requires only linear space, supports fast incremental updates, and avoids full recomputation as the window slides. We provide analytical guarantees on correctness, running time, update efficiency, and space complexity, demonstrating its scalability and low-latency performance for high-throughput streams.

We propose an **efficient and optimal** algorithm `BFair-ReOrder` to the **Continuous Stream Reordering** problem, which incorporates both the current window and the landmark items to maximize the number of *unique fair blocks* (where two blocks are considered unique if they have at least one uncommon item). Each fair block must satisfy specified group-wise representation constraints, and the goal is to maximize the number of such blocks not only within the current window but also across future windows defined by landmark positions. The key idea of the algorithm is to construct a specific permutation of the content, called an *isomorphic stream* (see details in Sec. 4), which satisfies the fairness requirement consistently within its blocks. We present theoretical analyses showing optimality and computational overhead.

We conduct extensive experiments using four large-scale real-world datasets by simulating streaming environment. The results align with our theoretical analysis, confirming the optimality of our solutions. The proposed framework processes queries in fractions of milliseconds, achieving an average throughput of 30,000 queries per second (depending on parameters), with minimal memory usage and high update efficiency. As no existing work addresses our problem setting, we design appropriate baselines. Our framework outperforms them by several orders of magnitude. To summarize, the paper makes the following contributions:

- **Block-level Fairness Model.** We propose a fairness model that partitions each sliding window into multiple blocks, enforcing localized group-fair exposure. This generalizes window-level fairness and offers a flexible alternative to prefix-based notions.
- **Continuous Fairness Monitoring and Stream Reordering.** We formalize the problem of *Monitoring Continuous Fairness* via a sketch-based algorithm `Monitor-BFair` along with the the *Continuous Stream Reordering Problem*, solved by our optimal algorithm `BFair-ReOrder` that constructs an *isomorphic stream* to maximize unique fair blocks.
- **Theoretical Guarantees.** We provide formal analysis of correctness, optimality, space, and time complexity. `B-FairQP` ensures linear space and fast incremental updates, while `Block-FairReOrder` is proven optimal and efficient.
- **Experimental Validation.** On four large-scale real-world datasets, our framework achieves sub-millisecond running times, throughput up to 30,000 queries/second, and minimal memory usage, outperforming baselines by several orders of magnitude.

2 Data Model and Formalized Framework

We first present our data model, followed by our proposed framework, based on which we formalize the studied problems. We start by introducing an example used throughout this section and based on the online content recommendation discussed in Section 1.

Example 2.1. A sliding window contains 15 items divided in 3 blocks (pages)- each item contains content generated by individuals with a protected attribute ethnicity. Wlog, imagine, there are 3 different ethic groups, show in Figure 2. There are two representative fairness constraints - shown in Table 3.

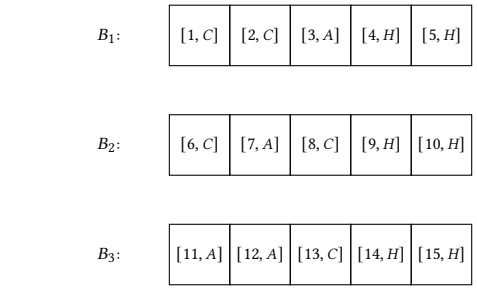


Figure 2: A sliding window of 15 items (contents) with 3 blocks, consisting of contents from three ethnic groups [*C* (Caucasian), *A* (Asian), *H* (Hispanic)].

Attr.	Value	C1	C2
Ethnicity	*C*	0.3	0.5
	A	0.3	0.2
	H	0.4	0.3

Figure 3: Two fairness constraints.

2.1 Data Model

Attribute denoting fairness. A protected attribute $\mathcal{A}$ has values ℓ possible values. Using Figure 1, *Ethnicity* is one of these attributes with three possible values, *C, A, H.*

Count based sliding window. A count-based sliding window is

a type of window in stream processing where computations are performed over the most recent fixed number of data items, and the window moves forward by a specified number of items (called the slide). Let W represent a sliding window with $|W|$ items, and the default slide is 1 (unless otherwise specified). Basically, it maintains the last $|W|$ items of the data stream and updates the result every time a new item arrives.

Using Example 2.1, each sliding window W contains 15 items.

Data item. A data item t in a sliding window W is a pair = $< i, p >$, where i is its order in W (key), and p represents any specific value of the protected attribute $\mathcal{A}$ that is, $p \in \ell$.

Using Example 2.1, the third video of the window is created by a contributor of Asian (A) ethnicity.

Blocks in a window. Let

$$W = \{t_1, t_2, \ldots, t_{|W|}\}$$

denote a sliding window containing $|W|$ items. We partition W into k equal-sized subintervals, each called a *block*, where the block size $s = \frac{|W|}{k}$.

Then, the q-th block B_q $(1 \le q \le k)$ is defined as

$$B_q = \{\, t_p \mid (q-1)s + 1 \le p \le qs \,\}.$$

Each block B_q contains exactly s consecutive items from the window:

$$B_1 = \{t_1, \ldots, t_s\}, B_2 = \{t_{s+1}, \ldots, t_{2s}\}, \ldots, B_k = \{t_{(k-1)s+1}, \ldots, t_{ks}\}.$$

Every item $t_j \in W$ thus belongs to exactly one block.

Using Example 2.1, each sliding window contains $k = 3$ blocks, each with $15/3 = 5$ items. Blocks B_1, B_2, B_3 start at item 1, 6, and 11, respectively. Figure 2 shows $\{B_1, B_2, B_3\}$ in window W.

The choice of block size s (or equivalently, the number of blocks k per window) is guided by the semantics of the target application. In streaming or recommendation settings, each block typically corresponds to the smallest granularity—such as a page of displayed items—over which fairness or representational balance is desired. The block granularity provides a natural control knob between fairness sensitivity and computational efficiency.

For any item $t_j \in W$, its block index is given by

$$\text{block}(t_j) = \left\lceil \frac{j}{s} \right\rceil.$$

Prefix, Suffix of a Block. For a block B_q starting at index i

$$B_q = [t_i, t_{i+1}, \ldots, t_{i+s-1}], \quad \text{for } 1 \le i \le |W| - s + 1.$$

its prefix of length j is:

$$\text{Prefix}_j(B_q) = [t_i, t_{i+1}, \ldots, t_{i+j-1}], \quad \text{for } 1 \le j \le s.$$

Its suffix of length j is:

$$\text{Suffix}_j(B_q) = [t_{i+s-j}, t_{i+s-j+1}, \ldots, t_{i+s-1}], \quad \text{for } 1 \le j \le s.$$

As an example, the first block B_1's prefix of length 3 are items $[t_1, t_2, t_3]$ and the suffix of length 2 are items $[t_4, t_5]$.

Landmark Items. Given the current window W, the landmark items consist of $|\mathcal{X}|$ additional elements stored immediately after the window, starting from position $|W| + 1$. Figure 4 shows 5 such additional items.

The landmark size is application-defined and serves as another design knob. In many streaming scenarios, a short wait for a few additional items beyond the current window is acceptable, as systems often employ slight buffering to refine fairness or diversity without perceptibly affecting latency. When no delay is permissible, reordering proceeds immediately using the current window, yielding the best fairness achievable with available data.

Unique Blocks. A block consists of s consecutive items, starting at item t and ending at item $t + s - 1$. Without loss of generality, two blocks are considered *unique* if each contains at least one item not present in the other. For example, the first block in the current window and the first block in the subsequent (slided) window are unique, as one contains items $[t_1, \ldots, t_5]$ while the other contains $[t_2, \ldots, t_6]$ after the slide.

Disjoint Blocks. Blocks that do not overlap with one another are called *disjoint blocks*. If each of these blocks individually satisfies the given fairness criterion, they are referred to as *disjoint fair blocks*. As an example block B_1, B_2, B_3 are fully disjoint.

Group Fairness Constraints. For the attribute $\mathcal{A}$ with ℓ different values, for each $p \in \cup_{i=1}^{l} \mathcal{A}_i$, $f(p)$ denotes its required proportion constraint, such that $\Sigma_{p=1}^{\ell} f(p) = 1$.

Table 3 shows two different group fairness constraints of the *Ethnicity* attribute.

Fair Sliding Window. Given a sliding window W and a desired proportion function $f(p)$ for each value p of the protected attribute $\mathcal{A}$, the window is considered *fair* if, in every block within W, the number of items with attribute value p lies within the range $[\lfloor f(p) \times s \rfloor, \lceil f(p) \times s \rceil]$ for all p. Otherwise, the window is deemed *unfair*. The floor and ceiling operations are used because $f(p) \times s$ may not be an integer, and these bounds ensure the proportion is rounded to the nearest feasible integer values.

Using *Constraint 1* listed in Table 3, this will require every block to have $[\lfloor .3 \times 5 \rfloor, \lceil .3 \times 5 \rceil]$ of data items of value C and value A ([1,2] data items) and $[\lfloor .4 \times 5 \rfloor, \lceil .4 \times 5 \rceil]$, i.e., 2 data items with protected attribute value H. Based on this requirement, the window shown in Figure 2 is fair.

This definition of fairness is consistent with the notion of *statistical parity* or *demographic parity* [5], appropriately adapted to scenarios where the order of items matters. It can also be viewed as a relaxed variant of *p-Fairness* [34], which enforces the proportionality constraint for every prefix of the ranked list. Finally, depending on the application requirements, one may incorporate an approximation factor ϵ in the definition, such that every block within a window satisfies

$$[\lfloor \epsilon f(p) \times s \rfloor,\ \lceil \epsilon f(p) \times s \rceil] \quad \forall\, p.$$

2.2 Proposed Framework

Given each sliding window and the specified fairness constraint, the framework (Refer to Figure 1) continuously monitors the windows. If the current window is *unfair*, the framework first checks whether it contains a sufficient number of data items so that the items in the window could be reordered to satisfy the constraints. This is done by counting the number of instances corresponding to each protected attribute value p, and verifying whether this count meets the required proportion for the entire window. Specifically, if the following condition holds:

$$\forall p \in \cup_{i=1}^{l} \mathcal{A}_i, \quad \sum_{i=1}^{|W|} \mathbb{1}[\{t_i(A) = p\} \geq (k \cdot \lfloor f(p) \cdot s \rfloor)]$$

Basically, for every attribute value $p \in \cup_{i=1}^{l} \mathcal{A}_i$, it checks if the total number of times p appears in the window W (i.e., the number of items i such that $t_i(A) = p$) is at least $k \cdot \lfloor f(p) \cdot s \rfloor$, where $f(p)$ denotes the proportionality of p, s is block size, and k is the number of blocks. In that case, the items in the current window can potentially be re-ordered to satisfy the fairness constraint. If the condition holds, then internal reordering can be easily performed within the current window. However, a more challenging scenario arises when the condition does not hold—this is the case we address closely in this work.

In the latter case, the framework pauses and reads an additional $|\mathcal{X}|$ landmark items. Each landmark item corresponds to a slide, collectively resulting in $|\mathcal{X}|$ additional sliding windows. The framework then considers both the items in the current window W and the landmark items $\mathcal{X}$, and performs reordering.

The ideal objective is to make both the current window and the $|\mathcal{X}|$ slided windows satisfy the fairness constraint. However, it is possible that even after incorporating the $|\mathcal{X}|$ landmark items, the combined set of data items may still lack sufficient instances of certain protected attribute values.

In such cases, the framework performs a special permutation within $W \cup \mathcal{X}$ with the goal of maximizing the total number of fair windows among the current window and the $|\mathcal{X}|$ slided windows.

After performing the permutation, the monitoring process resumes, and these two steps are repeated as needed until the entire stream has been processed.

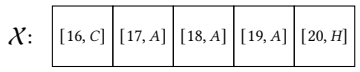


Figure 4: Five landmark items $\mathcal{X}$

Symbol	Explanation
W	Window with width $\|W\|$
B	Block in a window with width s
S	Sketch
k	Number of blocks in a window
$\mathcal{A}$	Protected attribute with cardinality ℓ
p	One value of $\mathcal{A}$
$\mathcal{X}$	Landmark items
$\mathcal{A}^i$	Protected attribute value of item i

Table 1: Key notations

2.3 Problem Definitions

Problem 1. **Continuous Fairness Monitoring** *For every sliding window W in the stream and a given fairness constraint, the window is deemed* fair *if each block $B \in W$ satisfies the constraint; otherwise, it is deemed* unfair.

Given the window in Figure 2 and the *Constraint 1* in Table 3, window W is fair. However, the *Constraint 2* in Table 3, window W is *unfair*.

Problem 2. **Continuous Stream Reordering using Landmark Items** *Given a window W and additional $|\mathcal{X}|$ landmark items, for a given fairness constraint, identify a reordering for data items in $W \cup \mathcal{X}$, so that it maximizes the total number of unique fair blocks in W and the slided windows generated by landmark items.*

Given the *Constraints-2* from Table 3, the minimum number of C instances required in window W of Figure 2 is calculated as $\lfloor 0.5 \times 5 \rfloor \times 3 = 6$. Since W contains only 5 instances of C, an additional 5 landmark items are read (as shown in Figure 4). These additional 5 items result in 5 more slided windows.

In total, this creates 16 unique blocks across the current window and the 5 slided windows. The objective is to perform reordering among the 20 available items (from W and the landmark items) such that the maximum possible number of these 16 blocks satisfy fairness constraints.

3 Continuous Fairness Monitoring

This section presents our technical solution for continuously and efficiently monitoring fairness constraints over each sliding window W. The framework consists of three main steps.

(1) **Sketch building:** It first computes the frequency distribution of the protected attribute $\mathcal{A}$ within the current window W, and stores this information in a lightweight data structure we refer to as the *Forward Sketch*.
(2) **Fairness check:** Given a fairness constraints, it then leverages the sketch to deem if the window is fair or not.
(3) **Sketch update:** As the window slides forward, the sketch is updated to reflect the changes, enabling continuous monitoring.

3.1 Preprocessing

At a high level, sketches [11, 19, 35] offer compact summaries of data distributions. In our setting, the goal is to capture the distribution of protected attribute values within a sliding window.

*Definition 3.1 (**Forward Sketch**).* A *Forward Sketch* `FSketch` S of width $|W|$ is a data structure associated with a window W, where each entry at index i stores a vector of length $(\ell - 1)$ representing the cumulative frequencies of $(\ell - 1)$ protected attribute values from the beginning of the window up to position i. The j-th entry of the vector corresponds to the j-th value of the protected attribute $\mathcal{A}$. Since the protected attribute $\mathcal{A}$ has ℓ unique values, the frequency of the remaining protected attribute value at index i can be inferred by subtracting the sum of the stored frequencies from i, the total number of items considered up to that point.

Consider the sketch S in Figure 5 that shows blocks B_1 and B_2 of the running example. The first bit of the vector is assigned to store the cumulative frequency of C and the second one is that of for A. As an example, the 4-th entry of S contains $[2, 1]$, denoting a total frequency of 2 of C and 1 of A from the beginning till the 4-th position of the window. Since $\mathcal{A}$ has three distinct values, the frequency of H could be inferred by subtracting each i from the cumulative frequency of C and A at position i. As an example, the cumulative frequency of H from the beginning upto position i is therefore $4 - (2 + 1) = 1$.

3.1.1 Sketch Construction Algorithm. Each entry i of the sketch S stores a vector of length $(\ell-1)$, representing the cumulative counts of the first $(\ell-1)$ protected attribute values from the start of the window up to position i. For $i > 1$, the sketch at index i copies the counts from the previous index $(i-1)$, and then increments the count corresponding to the j-th protected attribute value p if the item t_i has $\mathcal{A}^i = p$. If the encountered value is not stored explicitly in the vector, it does not do anything. This construction ensures that each sketch entry maintains a cumulative sum of the $\ell-1$ protected attribute values up to that point in the window. From there, the value of the protected attribute that is not stored could always be calculated. Algorithm 1 has the pseudocode.

Algorithm 1 Forward Sketch Construct - `FSketch`

Require: Window $W = \{t_1, t_2, \ldots, t_{|W|}\}$ of size $|W|$
Require: Set of protected attribute values $\mathcal{A} = \{p_1, p_2, \ldots, p_\ell\}$
Initialize sketch S as list of $|W|$ vectors, each of length $(\ell-1)$, with all entries set to 0
for $i = 1$ to $|W|$ **do**
  if $i > 1$ **then**
    $S[i] \leftarrow S[i-1]$ ▷ Copy counts from previous entry
  end if
  Let $p \leftarrow \mathcal{A}^{t_i}$ ▷ Protected attribute value of item t_i
  for $j = 1$ to $(\ell-1)$ **do**
    if $p = p_j$ **then**
      $S[i][j] \leftarrow S[i][j] + 1$
      break
    end if
  end for ▷ If $p = p_\ell$, no update is made
end for
return S

Theorem 3.2. *Algorithm* `Construct-Sketch` *(Algorithm 1) takes* $O(|W| \times \ell)$ *to run and takes* $O(|W| \times \ell)$ *space.*

Proof. From Algorithm 1, it is evident that each entry in the sketch requires $O(\ell)$ time to compute and $O(\ell)$ space to store, as it maintains information for $\ell-1$ protected attribute values. Since the outer loop iterates over the entire window of size $|W|$, the overall time and space complexity of the algorithm is $O(|W| \times \ell)$. □

3.1.2 Sketch Update. During the first time of creation, the sketch initializes by tracking $(\ell-1)$ protected attribute values, starting from the first index of the first window. This ensures that the earliest data items are captured first. The update ensures that the sketch accurately represents the most recent window of data. When window W is slided over, *the sketch shifts all entries one position to the left.* The last entry in the sketch is then updated considering the last but one entry and with the protected attribute value of the newly added item. This way, the sketch remains synchronized with the current window and continues to provide an up-to-date, compact summary of the protected attributes over time.

Consider the sketch in Figure 5 again. When the window slides over by one item, the 11-th item comes in. All entries in the previous sketch (refer to Figure 6) is now moved one position to the left, and the last entry of the sketch is now updated.

Lemma 3.3. *The sketch takes* $O(\ell)$ *time to update.*

Proof. Each vector is stored in a dynamic array that typically takes only a constant time to remove the first element of the sketch. A new vector is appended at the end, whose amortized cost is $O(\ell)$ time. □

S:	[1, 0]	[2, 0]	[2, 1]	[2, 1]	**[2,1]**	[3, 1]	[3, 2]	[4, 2]	[4, 2]	**[4,2]**

Figure 5: Sketch of blocks B_1 and B_2 in Figure 2

W:	[2, C]	[3, $\mathcal{A}$]	[4, $\mathcal{H}$]	[5, $\mathcal{H}$]	[6, C]	[7, $\mathcal{A}$]	[8, C]	[9, $\mathcal{A}$]	[10, $\mathcal{H}$]	[11, $\mathcal{A}$]
S:	[2, 0]	[2, 1]	[2, 1]	[2, 1]	[3, 1]	[3, 2]	[4, 2]	[4, 2]	[4, 2]	[4, 3]

Figure 6: Sketch maintenance with one item slide

3.2 Fairness Checking

Next, we discuss Algorithm `Monitor-BFair` that checks if a window is fair or not. The innovation of the algorithm is that it only needs to check at most the number of blocks k number of entries in the current sketch S, and returns the correct answer always. In some cases, it is able to terminate early, especially when it already has found violation of the fairness constraints. Specifically it only checks those entries of the current sketch S that represents the last entry of each block.

To evaluate the first block of size s, Algorithm `Monitor-BFair` first examines the s-th entry of the sketch S, which contains cumulative counts for the first $\ell-1$ protected attribute values. The count for the final (implicit) attribute is computed by subtracting the sum of the stored counts from item number. The fairness constraint is verified by checking whether all of these $\ell-1$ counts—and the inferred count of the ℓ-th attribute—satisfy the specified requirements.

For each subsequent block, the algorithm computes the difference between two sketch vectors : specifically, the last entry of the previous block and that of the current block. This difference yields the frequency of each protected attribute value within the current block. The process repeats for all blocks, continuing until either a block violates the fairness constraints—causing the algorithm to terminate early and return *No*—or all blocks pass the checks, in which case `Monitor-BFair` returns *Yes*. Algorithm 2 presents the pseudocode.

Consider Figure 5 and Constraints 1 from Table 3. The window W contains 10 elements, which `Monitor-BFair` partitions into two blocks, B_1, B_2, each of size $s = 5$. The algorithm starts by examining the sketch at the end of the first block, i.e., at index 5, where the sketch holds the cumulative vector $[2, 1]$. This indicates that within B_1, there are 2 instances of C and 1 instance of A, implying the remaining 2 items correspond to H. The block satisfies the first fairness constraint. Next, the algorithm proceeds to the end of the second block, at index 10. It computes the frequency vector for

B_2 by subtracting the sketch at index 5 from that at index 10. The resulting vector reveals that B_2 contains 2 instances of C, 1 instance of A, and 2 instances of H. This also satisfies the specified fairness requirements. This process continues for all three blocks. Finally, algorithm concludes that the window to be *fair*.

On the other hand, consider Figure 5 and Constraints 2 from Table 3. Its first two blocks would be deemed fair, but the third block will not, hence, the window is *unfair*.

Algorithm 2 `Monitor-BFair`:Fairness Checking Algorithms

Require: Sketch S with $|W|$ entries indexed from 1 to $|W|$;
Require: Block size s, number of blocks k;
Require: Fairness proportions $f(p)$ for each $p \in \{1, \ldots, \ell\}$
Ensure: **Fair** if all blocks satisfy fairness constraints; **Unfair** otherwise For$b \leftarrow 1$ to k

$curr \leftarrow b \cdot s$
if $b = 1$ **then**
  $countVec \leftarrow S[curr]$
else
  $prev \leftarrow (b-1) \cdot s$
  $countVec \leftarrow S[curr] - S[prev]$
end if
$total \leftarrow s$
$lastCount \leftarrow total - \sum_{j=1}^{\ell-1} countVec[j]$
Construct $fullVec[1 \ldots \ell] \leftarrow (countVec[1 \ldots \ell - 1], lastCount)$
for $p \leftarrow 1$ to ℓ **do**
  $minReq \leftarrow \lfloor f(p) \cdot s \rfloor$
  $maxReq \leftarrow \lceil f(p) \cdot s \rceil$
  if $fullVec[p] < minReq$ **or** $fullVec[p] > maxReq$ **then**
    return Unfair
  end if
end for
return Fair

Lemma 3.4. *Worst case running time of* `Monitor-BFair` *is* $O(k \times \ell)$

Proof. At the worst case, `Monitor-BFair` needs to process all k blocks, each requiring $O(\ell)$ time. Therefore, it overall will take $O(k \times \ell)$ time. □

Lemma 3.5. *Best case running time of* `Monitor-BFair` *is* $\Omega(\ell)$

Proof. At the best case, `Monitor-BFair` finds fairness violation in the first block itself, requiring $\Omega(\ell)$ time to declare the block *unfair* and terminate. □

Lemma 3.6. *Algorithm* `Monitor-BFair` *is optimal.*

Proof. Let W be a window of size $|W| = k \cdot s$, and let the fairness query specify desired proportions $f(p)$ for each protected attribute $p \in \{1, \ldots, \ell\}$. The goal of the query is to verify whether, in every block B_i of size s (for $1 \le i \le k$), the number of occurrences of each attribute p lies within the range $[\lfloor f(p) \cdot s \rfloor, \lceil f(p) \cdot s \rceil]$.

The sketch S stores cumulative counts of $\ell - 1$ protected attributes at each index $1 \le i \le |W|$. For any block B_i, the algorithm computes the frequency of each attribute as follows:

- For the first block B_1, it uses the cumulative count stored at position s to compute the frequency of the first $\ell - 1$ attributes in B_1.
- The frequency of the ℓ-th attribute is computed by subtracting the sum of the other $\ell - 1$ counts from the block size s.
- For each subsequent block B_i ($i > 1$), the algorithm computes the difference between the sketch values at positions $i \cdot s$ and $(i-1) \cdot s$, which yields the frequency of the first $\ell - 1$ attributes in B_i.
- Again, the frequency of the ℓ-th attribute is inferred as above.

In each block, the algorithm compares the frequency of every attribute value p to its allowed range. If a single attribute falls outside the allowed range in any block, the algorithm terminates early and correctly returns **No**. Otherwise, if all blocks satisfy the fairness constraints, it returns **Yes**. Since the sketch correctly captures cumulative counts and the algorithm faithfully reconstructs the per-block frequencies and verifies them against the fairness bounds, it always returns the correct answer.

□

4 Continuous Stream Reordering

Consider fairness **Constraints 2** again, there are three protected attribute values: C, A, H. Two distinct fairness requirements are specified:

Case 1: The requirement is $[2, 1, 2]$, meaning each block must contain 2 elements with value C, 1 with A, and 2 with H.
Case 2: The requirement is $[3, 1, 1]$.

A block is considered fair if it satisfies either of these criteria. Since the original block B_3 does not meet either requirement, 5 additional landmark items are read (Figure 4). The updated input stream, including these landmark items, now consists of 6 elements with value C, 7 with A, and 7 with H. We propose an algorithm, `BFair-ReOrder` , which takes these 20 elements and rearranges them to maximize the number of uniquely fair blocks.

Let $n = |W| + |\mathcal{X}|$ and $InStm = [ib_1, ib_2, \ldots, ib_n]$, be the stream under consideration with $|W|$ regular and $|\mathcal{X}|$ landmark items. The objective is to permute the elements into a new sequence $OutStm = [ob_1, ob_2, \ldots, ob_n]$ to maximize the number of unique fair blocks. Denote by $fb(Stm)$ the number of unique fair blocks in stream Stm. Then the problem can be formulated to maximize $OutStm = \arg\max_{Stm} fb(Stm)$.

4.1 Algorithm `BFair-ReOrder`

Algorithm `BFair-ReOrder` has an outer loop that takes each valid combination of protected attribute counts and run a subroutine (Subroutine `MaxReOrder`) to reorder and create the maximum number of unique fair blocks with that. For **Constraints 2**, there are two such combinations for $[C, A, H]$: $[2, 1, 2]$ and $[3, 1, 1]$, and it repeats the subroutine for each and outputs the one that maximizes the number of fair unique blocks at the end.

We begin by introducing key concepts used in the design of the Subroutine `MaxReOrder`.

Definition 4.1 (Isomorphic Blocks and Isomorphic Block Count). A set of disjoint blocks is said to be *isomorphic* if each block contains the same ordered sequence of protected attribute values. The total

number of such blocks in a stream is called the *Isomorphic Block Count (IBC)*. Under a given p-fairness constraint, if one isomorphic block satisfies the fairness condition, then all blocks in the set are considered fair.

Assume the input stream consists of ℓ distinct attribute values. Let $v_j \geq 1$ denote the number of elements with value $p \in \cup_{i=1}^{\ell} \mathcal{A}_i$ required by the fairness constraint, such that $\sum_{j=1}^{\ell} v_j = s$. Let V_j be the total number of elements of value p in the input stream. Then, the number of *isomorphic blocks* (IBC) that satisfy the given fairness requirement can be computed as:

$$\text{IBC} = \min\left\{\text{IBC}_j \mid 1 \leq j \leq \ell\right\}, \quad \text{where IBC}_j = \left\lfloor \frac{V_j}{v_j} \right\rfloor.$$

For example, in **constraints 2**, **Case 1:**

$$\left\lfloor \frac{6}{2} \right\rfloor = 3, \quad \left\lfloor \frac{7}{1} \right\rfloor = 7, \quad \left\lfloor \frac{7}{2} \right\rfloor = 3.$$

Thus, the total number of fair isomorphic blocks is:

$$\text{IBC} = \min(3, 7, 3) = 3.$$

For **Case 2:**, $\text{IBC} = \min(2, 7, 7) = 2$

Definition 4.2 (Isomorphic Stream and Extended Isomorphic Stream). A stream is called an *isomorphic stream* if it is entirely composed of isomorphic blocks—that is, each block follows the same fixed composition pattern. If such a stream is extended by a prefix of this block pattern, the resulting stream is called an *extended isomorphic stream*. This additional segment is referred to as the *extended prefix (EP)*, and its length is called the *extended prefix length (EPL)*.

Given a stream of length n and a fairness requirement specifying a block size s, if $IBC \cdot s = n$, the stream can be fully partitioned into IBC isomorphic blocks, or it can be reordered into an isomorphic stream.

If $IBC \cdot s < n$, the remaining $n - IBC \cdot s$ items cannot form a complete isomorphic block. These leftover elements may form an extended prefix EP. The length of the extended prefix is given by:

$$EPL = \sum_{i=1}^{\ell} EP_i, \quad \text{where } EP_i = \min\left(v_i,\ V_i - IBC \cdot v_i\right)$$

Here, ℓ is the number of distinct values (e.g., demographic groups) in the protected attribute; v_i is the required count of group i per fair block (from the fairness requirement); V_i is the total number of elements from group i in the entire stream; EP_i is the number of group i elements assigned to the extended prefix; IBC is the number of complete isomorphic blocks.
Note: $EPL < s$, since the extended prefix cannot complete a full block on its own.

Consider the fairness requirement of $[2, 1, 2]$ for C, A, and H, where $C = 6, A = 7, H = 7$.

The block size is $s = 5$, and since $IBC = 3$, we calculate: $EP_1 = \min(2,\ 6 - 3 \cdot 2) = 0, EP_2 = \min(1,\ 7 - 3 \cdot 1) = 1, EP_3 = \min(2,\ 7 - 3 \cdot 2) = 1$, which gives: $EPL = EP_1 + EP_2 + EP_3 = 0 + 1 + 1 = 2$. So the extended prefix is $EP = [A, H]$ (any permutation is valid). To complete the isomorphic block pattern, use the remaining counts $v_i - EP_i$:

$$\begin{aligned} C: &\quad 2 - 0 = 2 &&\Rightarrow [C, C] \\ A: &\quad 1 - 1 = 0 &&\Rightarrow [] \\ H: &\quad 2 - 1 = 1 &&\Rightarrow [H] \end{aligned}$$

One valid isomorphic block pattern is: $[A, H, C, C, H]$. We can now construct the extended isomorphic stream:

$$[A, H, C, C, H,\ A, H, C, C, H,\ A, H, C, C, H,\ A, H]$$

This consists of three full isomorphic blocks and a prefix of length 2, achieving optimal block-wise fairness. The 3 leftover As can be appended at the end to complete the stream.

To summarize, Subroutine `MaxReOrder`(see Subroutine 3) begins by computing IBC, the number of isomorphic blocks that can satisfy the given fairness requirement. If no block can satisfy the constraint, the algorithm simply returns the original stream.

If $IBC \cdot s = n$, where s is the block size and n is the total stream length, then the entire output stream can be constructed using exactly IBC isomorphic blocks, forming an *isomorphic stream*. In this case, the algorithm returns the built isomorphic stream directly.

If $IBC \cdot s < n$, the remaining elements—those that do not fit into the full isomorphic blocks—can be used to form an *extended prefix* EP. The extended stream is constructed by concatenating the IBC isomorphic blocks with this extended prefix. This stream contains the maximum number of fair blocks possible under the given constraints (see Lemma 4.3). Any leftover elements that are not part of the isomorphic blocks or the extended prefix are appended to complete the stream.

Algorithm 3 Subroutine `MaxReOrder`

Input: Input stream $InStm$ of length $|W| + \mathcal{X}$, P-fair constraint $[v_1, \ldots, v_\ell]$
Output: Reordered stream $OutStm$
Compute the number of fair isomorphic blocks IBC
if $IBC < 1$ **then**
    return $InStm$
end if
if $IBC \cdot s = n$ **then**
    Construct IBC fair isomorphic blocks based on the given P-fair constraint
    $OutStm \leftarrow$ isomorphic stream formed by the IBC fair isomorphic blocks
    return $OutStm$
end if
Build the extended prefix EP
Complete the Isomorphic block pattern using $v_i - EP_i$
Construct an extended stream by concatenating the IBC fair isomorphic blocks and EP
$OutStm \leftarrow$ extended stream + remaining elements
return $OutStm$

4.2 Optimality and Running Time Analysis

Lemma 4.3 (Optimality of Isomorphic and Extended Isomorphic Streams).

Proof. Let the stream have length n and block size s. The number of possible unique blocks of size s in the stream is $n - s + 1$.

We say a stream is *optimal* if all these blocks are *fair*—that is, it contains the maximum possible number of unique fair blocks.
An *isomorphic stream* consists of IBC = $\frac{n}{s}$ disjoint, identical fair blocks of size s. Because each overlapping block (i.e., any block of s consecutive elements) is composed of suffixes and prefixes of these identical fair blocks, every such block also satisfies the fairness condition. Therefore, all $n - s + 1$ unique blocks are fair, and the stream achieves optimality.
An *extended isomorphic stream* includes a carefully constructed prefix EP appending the isomorphic blocks. This prefix is designed such that any overlapping block between EP and the last isomorphic block is fair. Since all blocks within the isomorphic part are also fair, the entire stream contains $n - s + 1$ fair blocks. Thus, the extended isomorphic stream is also optimal.

□

Theorem 4.4 (Algorithm `BFair-ReOrder` is optimal).

Proof. If $IBC \cdot s = n$, the stream is an isomorphic stream, and the result follows from Lemma 4.3.

If $IBC \cdot s < n$, the extended stream includes $IBC \cdot s + EPL$ elements, which contribute $(IBC \cdot s + EPL - s + 1 = (IBC - 1) \cdot s + EPL + 1$ fair blocks. Based on the construction method, the remaining elements cannot complete another fair block or contribute to overlaps of any fair block. Thus, this is the maximum number of fair blocks possible. Subroutine `MaxReOrder` finds that optimality for each valid combination of protected attribute values and Algorithm `BFair-ReOrder` checks this for all possible combinations and outputs the one that maximizes it. □

Running time. Subroutine `MaxReOrder` takes at most $O(n)$ ($n = |W| + |\mathcal{X}|$) time. At the worst case, `MaxReOrder` needs to be run 2^ℓ times. Therefore, Algorithm `BFair-ReOrder` takes In $O(n \times 2^\ell)$ time. The space complexity is dominated by queue size, $O(n)$.

5 Experimental Evaluations

We present experimental analyses that focus on three aspects: (a) the effectiveness of the fairness model in ensuring fairness of block-level group, (b) the effectiveness of fairness checking and reordering solutions, and (c) the effectiveness and cost of the pre-processing framework. Code and data are available on [1].

5.1 Experimental Setup

Streaming environment. The streaming environment is simulated using Confluent Kafka Python (CKP), the official Python client for Apache Kafka. A single-broker exposes an external listener to receive data from a host-based producer, which streams JSON-formatted dataset records to a Kafka topic at a controlled rate.
The CKP-based consumer polls the broker, maintains an in-memory buffer of the latest $|W|$ messages, and forms a sliding window W for preprocessing and fairness checking.

Hardware and Software. All experiments are conducted on an HP-Omen 16-n0xxx (Windows 11 Home v10.0.26100, Python 3.9.13) with an 8-core AMD Ryzen 7 6800H CPU and 16 GB RAM. A single-node Kafka cluster (v3.7.0) and ZooKeeper (both from Confluent Platform 7.8.0) run in Docker (v4.41.2), simulating the streaming environment. Results are averaged over 10 runs.

Datasets. The experiments are conducted using 4 real world datasets (Table 2 has further details).
Hospital [8]: Contains over 10,000 patient admissions from a tertiary care hospital in India (April 2017–March 2019), sorted by admission date. The "AGE" attribute was quantile-binned into five bins.
Movies [16]: Includes 33M+ ratings from 200K+ users on 87,585 movies. Genres were mapped to 2 and 3 "moods" (dark/light/neutral) category, using joined movie and rating tables.
Stocks [27]: Tracks daily NASDAQ prices up to April 2020. Apple Inc.'s price changes were computed as percentage differences between open and close, binned into five equal-sized bins. Trading volume was binned into 2 and 3 categories.
Tweets[12]: Contains 16M tweets (April–May 2009). Sentiment analysis[25] was applied to assign one of five polarity-based sentiment categories.

Baselines. We note that *there is no related work that could be adapted to solve the two problems studied in the paper (see Section 6 for further details).* Therefore, we design our own baselines.
Backward Sketch or *BSketch*. This is designed to compare against the window preprocessing technique discussed in Section 3.1 referred to as `FSketch`. We call this *BSketch* as this starts constructing the sketch backwards. Basically, at index i it captures the cumulative counts of the $\ell - 1$ protected attribute values starting i-th position till the end of the window. Naturally, for every new window, the sketch must be reconstructed from scratch because all items in sketch will need fresh update (not just shifts unlike forward sketch).
BSketchQP For monitoring continuous fairness that uses *BSketch* for preprocessing. This is designed to compare against `Monitor-BFair` proposed in Section 3.2.
BruteForce that enumerates all possible reordering and selects the one that maximizes the number of unique fair blocks to compare against `BFair-ReOrder` discussed in Section 4. Naturally, *BruteForce* is computationally intensive and does not scale at all.

Measures. For fairness analysis, we compare the percentage of fair blocks produced by our algorithms compared to the original stream. Then, we present a small qualitative study that compare block level fairness with window level fairness based on user satisfaction. We also evaluate the optimality of `BFair-ReOrder`.
We measure the average fairness checking time, 90% tail latency (i.e., the time within which 90% of the check is complete, while 10% take longer), throughput (number of fairness constraints answered per unit time), and memory consumption.
For preprocessing analysis, we report sketch construction and update time, memory consumption, as well as 90% tail latency of preprocessing.

Fairness constraints and Parameters. We vary window size $|W|$, block size s, cardinality of the protected attribute ℓ, landmark size $|\mathcal{X}|$, as appropriate. Fairness constraints are generated from the proportion of attribute values present in the respective datasets using the protected attributes discussed in Table 2; one protected attribute at a time. The default parameter values are $|W|$=1000, s=25, $|\mathcal{X}|$=100 and $\ell = 3$.

Datasets used	Protected attribute(s)	Records
Hospital	Gender, Outcome, Age	10,101
Twitter	Sentiments	16,000,000
MovieLens	Ratings, Moods (2,3)	33,832,161
Stock	% price change, volume of trade	9,908

Table 2: Datasets & statistics

Summary of results. Our experimental analysis reveals several key findings. First, our qualitative study demonstrates that the users unanimously prefer our proposed fairness model, as opposed to enabling group fairness only on the overall window. Second, the empirical results are consistent with our theoretical guarantees for the proposed algorithms used in preprocessing, reordering, and fairness checking. Third, our framework outperforms all designed baselines by 2–3 orders of magnitude in terms of efficiency. Fourth, it effectively enforces fairness at the block level, substantially increasing the percentage of fair blocks when `BFair-ReOrder` is applied. Additionally, the cost of fairness monitoring remains minimal in both time and memory: `Monitor-BFair` answers queries in fractions of a millisecond, and `BFair-ReOrder` scales linearly with the input stream size. Overall, the framework is accurate and scalable across a wide range of parameters.

5.2 Fairness Analyses & Qualitative Study

Datasets	Parameter setting	Before Reorder	After Reorder
Hospital (s=25, |X| = 100, l = 5)	|W| = 200	0%	62%
	|W| = 1000	8%	95%
	|W| = 2000	1%	93%
Tweets (|W|=1000, |X| = 500, l = 5)	s = 25	0%	32%
	s = 100	0%	87%
	s = 250	0%	96%
Movies (|W|=1000, s = 250, |X| = 500)	l = 2	2%	98%
	l = 3	0%	99%
	l = 5	0%	21%

Table 3: Fairness achieved by `BFair-ReOrder`

5.2.1 Fairness Analyses. We compare our proposed reordering algorithm `BFair-ReOrder` with `BruteForce` to empirically evaluate our theoretical claim. We observe that `BFair-ReOrder` returns the same number of unique fair blocks as that of `BruteForce` corroborating its optimality in Figure 7.

We vary window size $|W|$, block size s, and cardinality ℓ of different datasets and present the percentage of unique blocks that become fair after `BFair-ReOrder` is applied on the original stream. In Table 3 presents a subset of representative results. A general observation is, `BFair-ReOrder` significantly improves the fairness of the data stream. In general, with increasing window size $|W|$, as well as block size s, fairness substantially improves, which is intuitive, because a larger window or block size increases the likelihood of more number of blocks satisfying fairness for the same constraints. On the other hand, with increasing cardinality, the group fairness requirement becomes more complex, leading to less percentage of fair blocks.

In Table 9, we vary $|\mathcal{X}|$ and measure the percentage of unique fair blocks after `BFair-ReOrder` is applied. The results indicate that with increasing $|\mathcal{X}|$, the percentage of unique fair blocks increases - this is intuitive, since with increasing landmark items, it becomes more likely to satisfy fairness constraints, thereby increasing the overall percentage.

5.2.2 Qualitative Study. We conduct a qualitative study with 10 graduate students to compare perceived recommendation quality under block-level vs. window-level fairness in two online content recommendation applications: (1) **Song playlists**, consisting of 15 tracks from three genres (pop, electronic, and country) displayed over three pages (blocks), and (2) **ML content**, comprising 15 YouTube videos across three types (tutorial, project, and explainer) displayed over three pages (blocks).

For each task, two recommendation orders—one ensuring per-page (block-level) and the other overall (window-level) representation—are presented in randomized order, with participants blinded to the model used. Participants answer three questions for each application: (1) What is your average satisfaction with the list generated by Model A? (2) What is your average satisfaction with the list generated by Model B? (3) Between Model A and Model B, which list do you prefer overall?

Table 4 summarizes the qualitative feedback from participants comparing block-level and window-level fairness models across the two applications. Overall, participants reported noticeably higher satisfaction with the recommendations produced by the block-level model (average satisfaction above 4.2) compared to the window-level model (average satisfaction around 3.0). A strong majority of users (approximately 89%) preferred the block-level ordering in both the song playlist and ML content scenarios. These results suggest that enforcing fairness at the block level yields recommendations that are perceived as both fairer and more satisfying to users, particularly in stream settings.

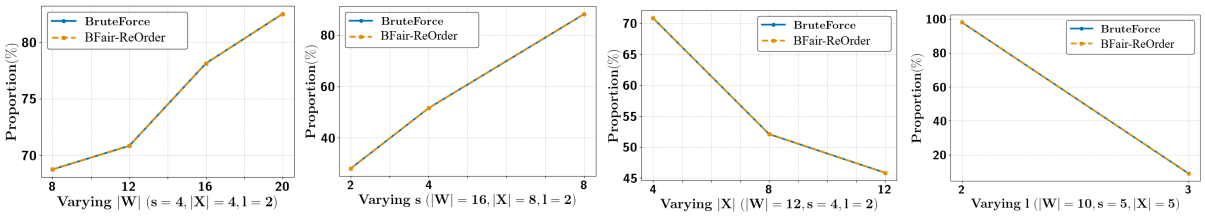
Application	Avg. Sat. (Block)	Avg. Sat. (Window)	% Prefer Blocks	% Prefer Windows
Song Playlist	4.22	3.00	89%	11%
ML Content	4.33	3.00	89%	11%

Table 4: User response comparison: block vs. window-level fairness.

5.3 Running Time of Fairness Algorithms

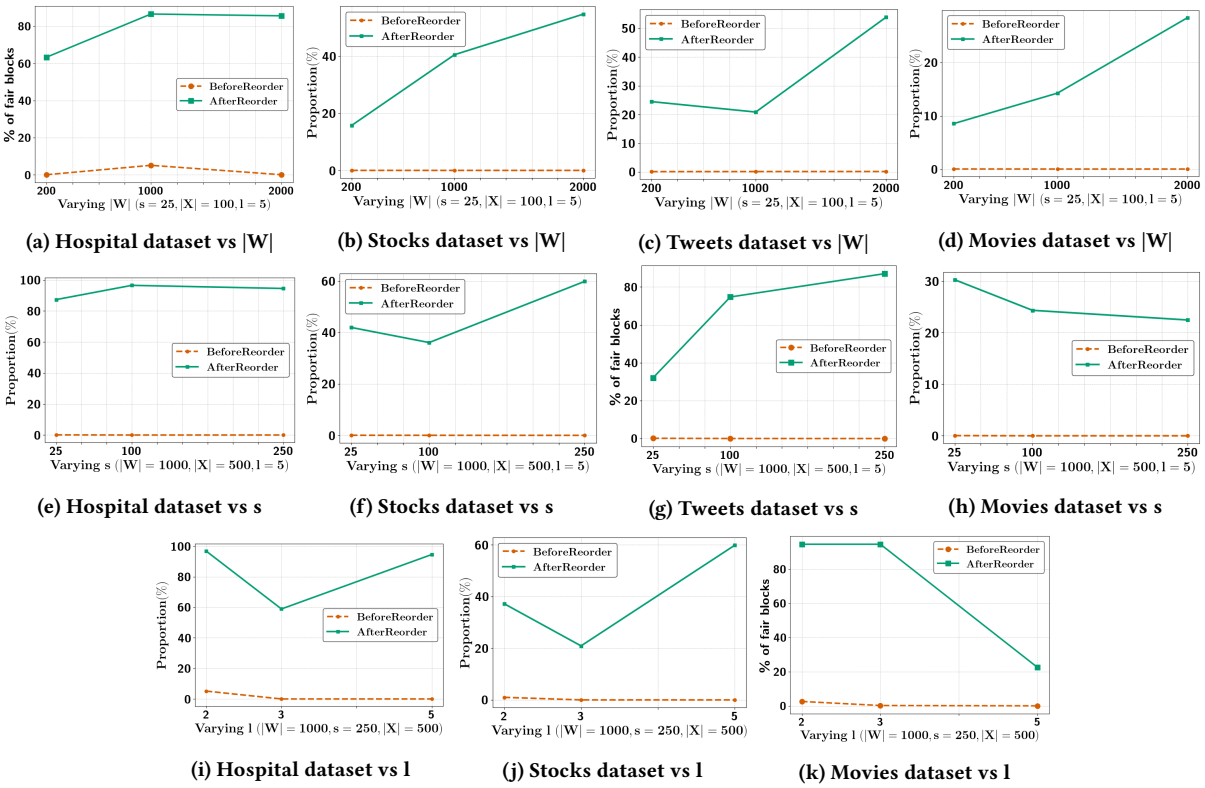
We first present the effectiveness of `Monitor-BFair` and `BFair-ReOrder` varying pertinent parameters. In Section 5.3.4, we separately compare implemented baselines with our solutions, exhibiting that the baseline is $2-3$ order of magnitude slower than ours.

5.3.1 Throughput. In these experiments, we measure the throughput (per second) of `Monitor-BFair` by varying $|W|$, s, and ℓ, considering both scenarios: one where re-ordering involving landmark items is required using `BFair-ReOrder`, and one where it is not.



(a) Varying Window size (b) Varying Block size (c) Varying Landmark items (d) Varying Cardinality

Figure 7: Qualitative Analysis: Performance against BruteForce



(a) Hospital dataset vs |W| (b) Stocks dataset vs |W| (c) Tweets dataset vs |W| (d) Movies dataset vs |W|

(e) Hospital dataset vs s (f) Stocks dataset vs s (g) Tweets dataset vs s (h) Movies dataset vs s

(i) Hospital dataset vs l (j) Stocks dataset vs l (k) Movies dataset vs l

Figure 8: Fairness analysis

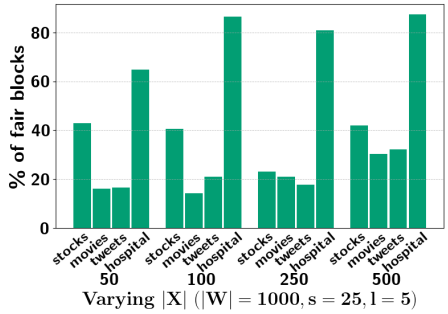
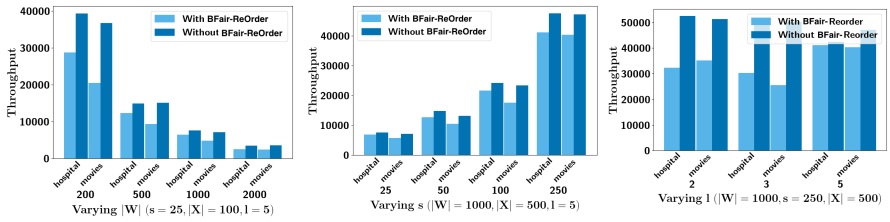


Figure 9: Effect of landmark on fairness

Figure 10a have these results and demonstrate that our algorithms exhibit very high throughout (as high as 40,000 at cases). As expected, `Monitor-BFair` incurs higher processing time when reordering is required, resulting in lower throughput, and vice versa. We also observe an inverse relationship between $|W|$ and throughput. This is indeed true as the window size increases $|W|$, this leads to processing higher number of blocks, leading to more time.

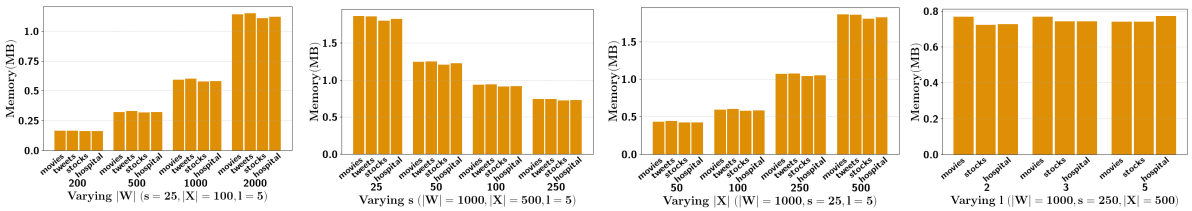
On the other hand, in Figure 10b, we observe the directly proportional relationship between s and throughput. Growing s decreases the number of blocks for a fixed $|W|$, thus decreasing the time taken for monitoring fairness and increasing the throughput.

The impact of ℓ is less pronounced (Figure 10c). While higher cardinality may increase running time, evenly distributed constraints



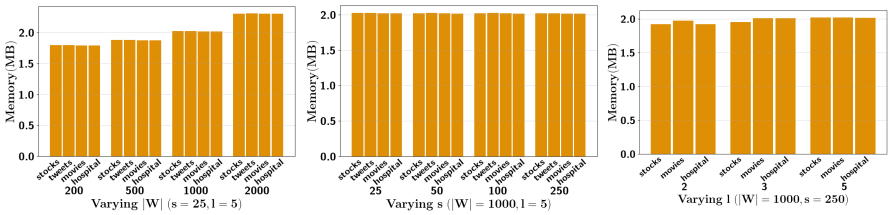
(a) Varying window sizes (b) Varying block sizes (c) Varying cardinality

Figure 10: Throughput with and without `BFair-ReOrder`



(a) Varying Window Size (b) Varying Block Size (c) Varying Landmark items (d) Varying Cardinality

Figure 11: Query Processing with BFair-ReOrder Memory Consumption over 5000 windows



(a) Varying Window Size (b) Varying Block Size (c) Varying Cardinality

Figure 12: Query processing without BFair-ReOrder Memory Consumption over 5000 read-only windows

often lead to more fair blocks, reducing reordering effort. Thus, overall running time stays stable with moderate ℓ, and throughput remains nearly constant without `BFair-ReOrder`.

5.3.2 Tail Latency. We report the 90th percentile tail latency of `Monitor-BFair` over 5,000 windows, both with and without algorithm `BFair-ReOrder`, as shown in Figure 13.

As shown in Figure 13a, tail latency increases with larger $|W|$. For instance, the time to process a single fairness constraints at times can exceed 10 ms when $W = 2000$. However, it is important to note that this represents the slowest 10% of cases and does not reflect the typical running time.

Figure 13b illustrates the expected decrease in tail latency with increasing s, consistent with earlier observations.

Similarly, Figure 13c shows the effect of increasing ℓ. Due to variations in the distribution of values across different $\mathcal{A}$, the tail latency does not exhibit a clear trend in this case.

Tail latency is significantly higher when `BFair-ReOrder` is needed inside `Monitor-BFair`, as reordering introduces substantial overhead compared to just monitoring. In contrast, the absence of reordering results in minimal delays, even for the slowest 10% of queries.

5.3.3 Running Time & Memory Consumption. Average running time is typically in the order of fraction of milliseconds, even when reordering is needed. Memory consumption of `Monitor-BFair` is negligible and consistent to the sketch memory consumption presented later.

5.3.4 Running Time compared to Baseline. We compare the runtime of `BFair-ReOrder` with that of `BruteForce` across varying values of $|W|$, s, and $|\mathcal{X}|$, as shown in Figure 14. Due to the extreme inefficiency of the `BruteForce` algorithm, the streaming environment automatically terminated the process (connection closed due to polling for over 5 minutes) when attempting experiments with window and landmark sizes above 24 items or cardinality beyond 3. Figure 14a shows the inefficiency of `BruteForce`: while comparable to `BFair-ReOrder` for $|W| \leq 16$, its runtime explodes beyond 10 s at $|W| = 20$, confirming its exponential complexity. `BruteForce` exhaustively searches all reorderings, making it highly sensitive to s (Figure 14b); larger s reduces blocks, increasing the chance of early fairness. Increasing $|\mathcal{X}|$ similarly extends runtime but with a gentler slope. Overall, `BFair-ReOrder` is over three orders of magnitude

faster. BSketchQP is consistently outperformed by Monitor-BFair, and those results are omitted for brevity.

5.4 Preprocessing Analyses

We present the preprocessing analyses of our solutions, and later in the section we present the comparison with appropriate baselines.

5.4.1 Sketch Construction & Update. We report the preprocessing time for 5,000 windows on the stocks and movies datasets in Figure 16, showing the time breakdown for sketch construction and updates, with and without BFair-ReOrder (please note if BFair-ReOrder is used the sketch needs to be constructed from scratch). Without BFair-ReOrder, sketch construction time is negligible, as it is built once and updated in the remaining windows. As shown in Figure 16a, preprocessing time grows with $|W|$ due to higher sketch construction cost, while update time remains stable. Varying s or ℓ has minimal impact (Figure 16b), showing robustness to block size and cardinality. At times, preprocessing can be slower without BFair-ReOrder, since batch memory access via landmark items (with fixed $|X| = 500$) and reduce overhead compared to loading new records per window.

5.4.2 Memory Requirement. Figure 18 shows average peak memory usage during preprocessing over 5,000 windows, reflecting the maximum memory used—including background processes—under varying parameters.
As expected, memory consumption of FSketch increases with $|W|$ (Figure 18a) due to larger sketches. It decreases with increasing s (Figure 18b), as larger blocks reduce the number of configurations needed during BFair-ReOrder, lowering memory usage. Similarly, memory usage increases with $|\mathcal{X}|$, not due to sketch size, but because reordering longer streams in BFair-ReOrder incurs greater background memory overhead. These experiments corroborate our theoretical analyses, exhibiting that propsoed FSketch is lean and bounded by window size.

5.4.3 Comparison with Baseline. We compare the preprocessing times of *BSketch* and *FSketch* in Figure 19 on the stocks dataset under varying parameters. *FSketch* consistently outperforms *BSketch* by an orders of magnitude, with the gap widening as window size increases (Figure 19a)—a result of Backward Sketch rebuilding the sketch from scratch for each window, unlike Forward Sketch, which supports efficient updates. As shown in Figures 19b and 19c, block size and cardinality have little impact, with minor variations attributed to platform-level fluctuations rather than the methods themselves.

5.5 Component-wise Running Time

We evaluate the average runtime of four core components — Sketch Construction, Sketch Update, BFair-ReOrder, and Monitor-BFair — on all the four dataset under varying $|W|$, s, and $|\mathcal{X}|$. BFair-ReOrder (Figure 21) incurs the highest cost, followed by Sketch Construction, while Sketch Update and Monitor-BFair remain consistently lightweight. Sketch Construction scales linearly with $|W|$ but is largely insensitive to s and $|\mathcal{X}|$. In contrast, BFair-ReOrder scales linearly with both $|W|$ and $|\mathcal{X}|$, with runtime slightly decreasing as s increases.

6 Related Work

To the best of our knowledge, no related work studies the problems we propose.

Group Fairness. Group fairness, initially studied in classification, ensures equitable treatment across demographic groups defined by protected attributes (e.g., race, gender, age), commonly measured via *demographic* or *statistical parity* [17, 21]. In data management, it has been extensively explored in ranking and recommendation [18, 22, 30, 32, 34], where fairness notions such as *top-k parity* [22] and *P-fairness* [34] ensure balanced group representation across results or prefixes. Extending these guarantees to *multiple* protected attributes remains computationally challenging [9, 20] due to the exponential growth of intersecting subgroups.
We generalize the notion of P-fairness *to every block (instead of every prefix), and initiate its study in the streaming settings.*

Data streams. Research on data streams has evolved from efficient processing to maintaining data quality under continuous updates. Foundational systems such as CQL [2], C-SPARQL [4], and window join algorithms [15, 24, 26] addressed scalable, memory-efficient stream handling. Subsequent work focuses on consistency and constraint management, with denial constraints [10] enabling logical validation and recent frameworks [28, 29] supporting real-time monitoring and adaptive schema enforcement. Existing works [23, 31] also propose automatic repair techniques to preserve constraint satisfaction during streaming updates.

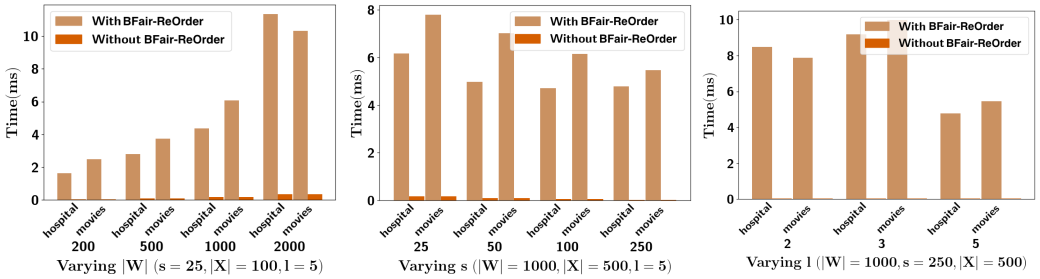
In contrast, we focus on enforcing fairness constraints within each block of the window, allowing only data rearrangement without modifying or imputing values. As a result, existing approaches on data quality and repair are not directly applicable to our setting.

Stream fairness. Research on data stream have recently started paying more attention to fairness. Baumeister et al. [6] checks fairness measures like demographic parity and equalized odds by estimating probabilities from live data streams. Wang et al. [33] address fairness in evolving data streams with *Fair Sampling over Stream (FS^2)*, a method that balances class distribution while considering fairness and concept drift. They also propose *Fairness Bonded Utility (FBU)*, a metric to trade-off between fairness and accuracy.

In contrast, we propose continuous fairness queries over data streams ensuring fairness within each block of the sliding window.

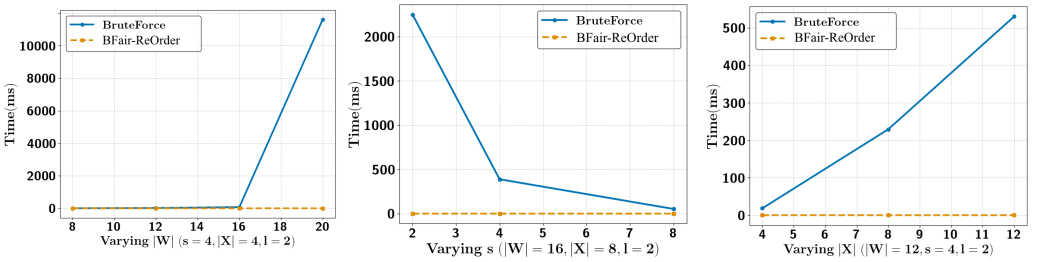
7 Conclusion

We initiate the study of group fairness on data streams, which enforces fairness at a finer granularity through block-level evaluation within sliding windows. This localized approach captures fairness violations more precisely and flexibly than traditional window-level methods. We propose a comprehensive computational framework that includes an efficient sketch-based monitoring structure and an optimal stream reordering algorithm, both designed to support continuous fairness monitoring and reordering in real time. Together, these contributions enable scalable, low-latency, and principled enforcement of group fairness in high-throughput streaming systems, offering strong theoretical guarantees and practical utility for dynamic, fairness-critical applications.
We are currently investigating how to extend the proposed framework to simultaneously handle multiple protected attributes (e.g., gender, ethnicity, and age). While we recognize the inherent computational intractability of this problem, our preliminary findings



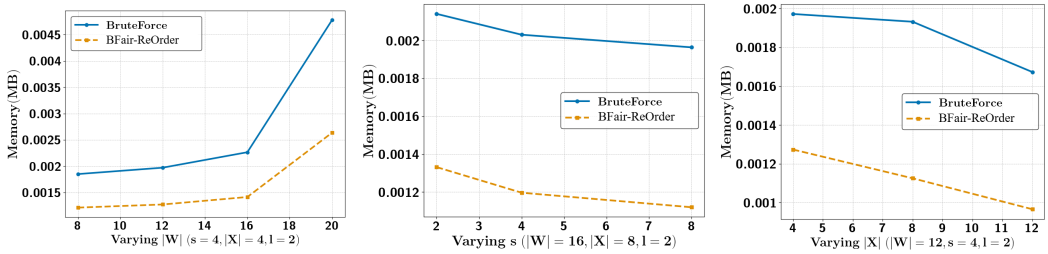
(a) Varying window sizes (b) Varying block Sizes (c) Varying cardinality

Figure 13: 90 percentile running time tail latency over 5000 windows



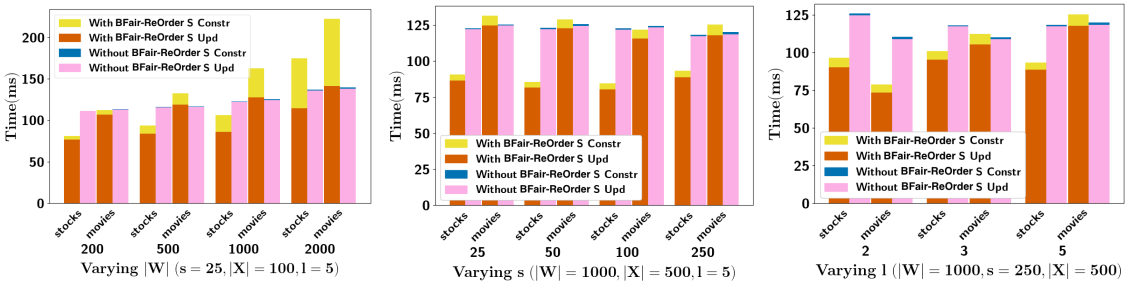
(a) Varying window sizes (b) Varying block sizes (c) Varying landmark items

Figure 14: `BruteForce` vs `BFair-ReOrder` running time



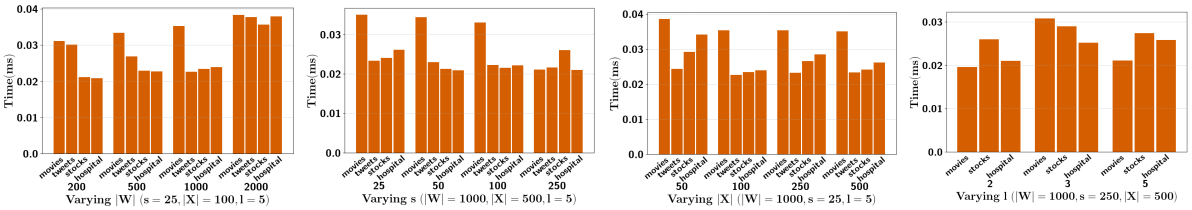
(a) Varying Window sizes (b) Varying Block sizes (c) Varying Landmark

Figure 15: BruteForce vs p-FairReOrder Memory Consumption across different parameters



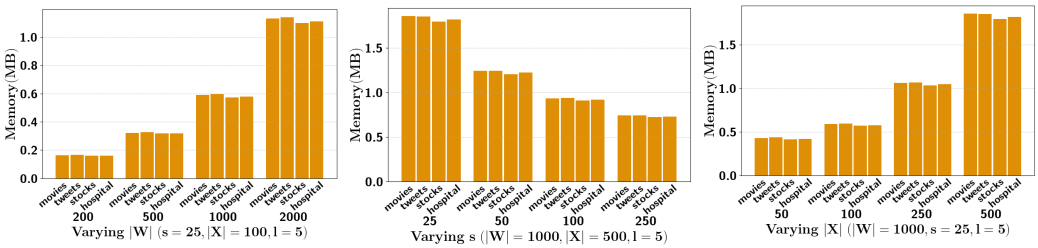
(a) Varying window sizes (b) Varying block sizes (c) Varying cardinality

Figure 16: Total preprocessing running time for 5000 windows



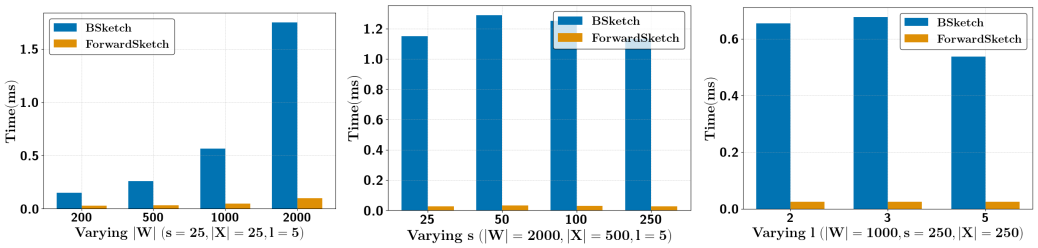
(a) Varying Window Size (b) Varying Block Size (c) Varying Landmark items (d) Varying Cardinality

Figure 17: 90 percentile preprocessing tail latency over 5000 windows



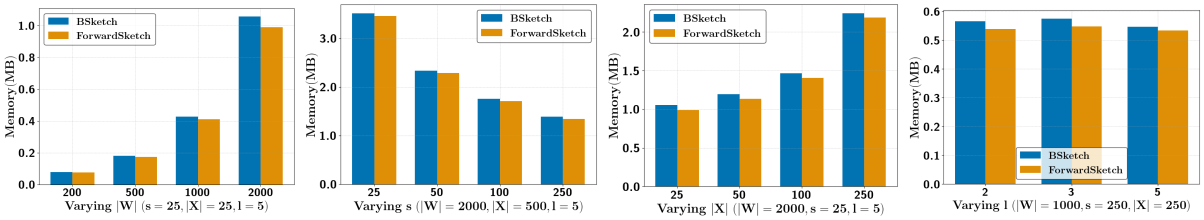
(a) Varying window size (b) Varying block size (c) Varying landmark items

Figure 18: Average peak memory consumption over 5000 windows



(a) Varying window size (b) Varying block size (c) Varying cardinality

Figure 19: Preprocessing time of `Backward Sketch` vs `Forward Sketch`



(a) Average preprocessing peak consumption for varying window sizes (b) Average preprocessing peak consumption for varying block sizes (c) Average preprocessing peak consumption for varying landmark items (d) Average preprocessing peak consumption for varying cardinality

Figure 20: BSketch vs ForwardSketch Memory Consumption across different parameters

indicate that the reordering problem in particular poses substantial challenges, as the current solution does not generalize to satisfying fairness across multiple attributes.

References

[1] 2025. Anonymous git. https://anonymous.4open.science/r/ProposesdFramework-B66B. Anonymous GitHub repository.

[2] Arvind Arasu, Shivnath Babu, and Jennifer Widom. 2006. The CQL continuous query language: semantic foundations and query execution. *VLDB J.* 15, 2 (2006), 121–142.

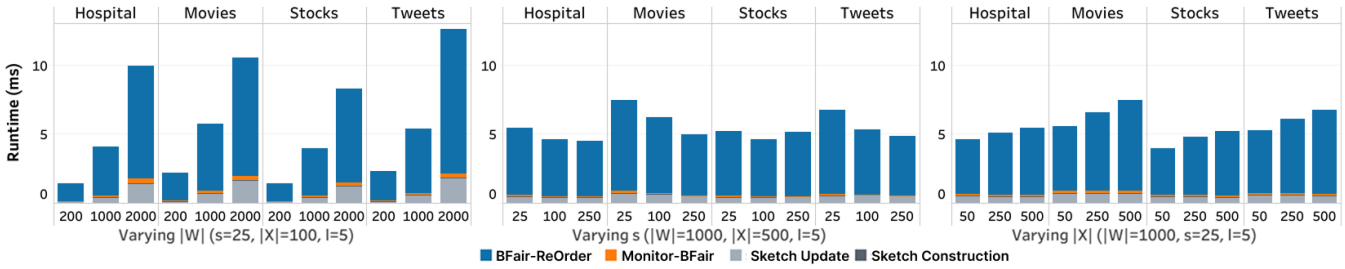


Figure 21: Component-wise Running Time

[3] Shivnath Babu and Jennifer Widom. 2001. Continuous queries over data streams. *ACM Sigmod Record* 30, 3 (2001), 109–120.

[4] Davide Francesco Barbieri, Daniele Braga, Stefano Ceri, Emanuele Della Valle, and Michael Grossniklaus. 2009. C-SPARQL: SPARQL for continuous querying. In *WWW*. ACM, 1061–1062.

[5] Solon Barocas, Moritz Hardt, and Arvind Narayanan. 2019. Fairness and machine learning: Limitations and opportunities. fairmlbook. org.

[6] Jan Baumeister, Bernd Finkbeiner, Frederik Scheerer, Julian Siber, and Tobias Wagenpfeil. 2025. Stream-Based Monitoring of Algorithmic Fairness. In *TACAS (1) (Lecture Notes in Computer Science, Vol. 15696)*. Springer, 60–81.

[7] Reuben Binns. 2020. On the apparent conflict between individual and group fairness. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency* (Barcelona, Spain). Association for Computing Machinery, New York, NY, USA, 514–524. https://doi.org/10.1145/3351095.3372864

[8] Sandeep Chandra Bollepalli, Ashish Kumar Sahani, Naved Aslam, Bishav Mohan, Kanchan Kulkarni, Abhishek Goyal, Bhupinder Singh, Gurbhej Singh, Ankit Mittal, Rohit Tandon, Shibba Takkar Chhabra, Gurpreet S. Wander, and Antonis A. Armoundas. 2022. An Optimized Machine Learning Model Accurately Predicts In-Hospital Outcomes at Admission to a Cardiac Unit. *Diagnostics* 12, 2 (2022). https://doi.org/10.3390/diagnostics12020241

[9] Felix S Campbell, Alon Silberstein, Julia Stoyanovich, and Yuval Moskovitch. 2024. Query Refinement for Diverse Top-k Selection. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[10] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering Denial Constraints. *Proc. VLDB Endow.* 6, 13 (2013), 1498–1509.

[11] Mayur Datar, Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 2002. Maintaining Stream Statistics over Sliding Windows. *SIAM J. Comput.* 31, 6 (2002), 1794–1813.

[12] FERNO2. 2020. training.1600000.processed.noemoticon.csv. https://www.kaggle.com/datasets/ferno2/training1600000processednoemoticoncsv.

[13] David García-Soriano and Francesco Bonchi. 2021. Maxmin-fair ranking: individual fairness under group-fairness constraints. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* (Virtual Event, Singapore). Association for Computing Machinery, New York, NY, USA, 436–446. https://doi.org/10.1145/3447548.3467349

[14] Lukasz Golab and M Tamer Özsu. 2003. Processing sliding window multi-joins in continuous queries over data streams. In *Proceedings 2003 VLDB Conference*. Elsevier, 500–511.

[15] Lukasz Golab and M. Tamer Özsu. 2003. Processing Sliding Window Multi-Joins in Continuous Queries over Data Streams. In *VLDB*. Morgan Kaufmann, 500–511.

[16] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (Dec. 2015), 19 pages. https://doi.org/10.1145/2827872

[17] Corinna Hertweck, Christoph Heitz, and Michele Loi. 2021. On the moral justification of statistical parity. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. 747–757.

[18] Nyi Nyi Htun et al. 2021. Perception of fairness in group music recommender systems. In *IUI*. 302–306.

[19] Piotr Indyk. 2006. Stable distributions, pseudorandom generators, embeddings, and data stream computation. *J. ACM* 53, 3 (2006), 307–323.

[20] Md Mouinul Islam et al. 2022. Satisfying complex top-k fairness constraints by preference substitutions. *PVLDB* (2022).

[21] Zhimeng Jiang, Xiaotian Han, Chao Fan, Fan Yang, Ali Mostafavi, and Xia Hu. 2022. Generalized demographic parity for group fairness. In *International Conference on Learning Representations*.

[22] Caitlin Kuhlman and Elke Rundensteiner. 2020. Rank aggregation algorithms for fair consensus. *Proceedings of the VLDB Endowment* 13, 12 (2020).

[23] Samuele Langhi, Angela Bonifati, and Riccardo Tommasini. 2025. Evaluating Continuous! eries with Inconsistency Annotations. (2025).

[24] Hyo-Sang Lim, Jae-Gil Lee, Min-Jae Lee, Kyu-Young Whang, and Il-Yeol Song. 2006. Continuous query processing in data streams using duality of data and queries. In *SIGMOD Conference*. ACM, 313–324.

[25] Steven Loria. 2018. textblob Documentation. *Release 0.15* 2 (2018).

[26] Kamesh Munagala, Utkarsh Srivastava, and Jennifer Widom. 2007. Optimization of continuous queries with shared expensive filters. In *PODS*. ACM, 215–224.

[27] Oleh Onyshchak. 2020. Stock Market Dataset. https://doi.org/10.34740/KAGGLE/DSV/1054465

[28] Vasileios Papastergios and Anastasios Gounaris. 2025. Stream DaQ: Stream-First Data Quality Monitoring. *CoRR* abs/2506.06147 (2025).

[29] Eduardo H. M. Pena, Fábio Porto, and Felix Naumann. 2024. Discovering Denial Constraints in Dynamic Datasets. In *ICDE*. IEEE, 3546–3558.

[30] Evaggelia Pitoura et al. 2022. Fairness in rankings and recommendations: an overview. *The VLDB Journal* (2022), 1–28.

[31] Shaoxu Song, Aoqian Zhang, Jianmin Wang, and Philip S. Yu. 2015. SCREEN: Stream Data Cleaning under Speed Constraints. In *SIGMOD Conference*. ACM, 827–841.

[32] Julia Stoyanovich, Bill Howe, and Hosagrahar Visvesvaraya Jagadish. 2020. Responsible data management. *Proceedings of the VLDB Endowment* 13, 12 (2020).

[33] Zichong Wang, Nripsuta Saxena, Tongjia Yu, Sneha Karki, Tyler Zetty, Israat Haque, Shan Zhou, Dukka Kc, Ian Stockwell, Xuyu Wang, Albert Bifet, and Wenbin Zhang. 2023. Preventing Discriminatory Decision-making in Evolving Data Streams. In *FAccT*. ACM, 149–159.

[34] Dong Wei et al. 2022. Rank aggregation with proportionate fairness. In *SIGMOD*.

[35] Li Yan, Nerissa Xu, Guozhong Li, Sourav S. Bhowmick, Byron Choi, and Jianliang Xu. 2022. SENSOR: Data-driven Construction of Sketch-based Visual Query Interfaces for Time Series Data. *Proc. VLDB Endow.* 15, 12 (2022), 3650–3653.